

Developer Manual

TMADJ Temple POS — Technical Reference

TMADJ Temple POS System

TMADJ Temple POS — Developer Manual

Technical reference for maintaining and extending the TMADJ Temple Management & POS system.

1. Overview & Tech Stack

Offline-first PWA for temple counter billing, hall bookings, devotee management, and reporting.

Layer	Technology
Frontend	React 18 + Vite 5, JavaScript (JSX, no TypeScript), React Router v6, Tailwind CSS, Lucide icons
Frontend state	React Context (<code>AppContext</code>) + local component state — no Redux/Zustand/TanStack Query
Offline / PWA	<code>vite-plugin-pwa</code> (Workbox service worker) + Dexie.js (IndexedDB) for local mirror + outbox queues
Backend	Node.js + Express 4, ES modules
Database access	Sequelize ORM → MySQL (<code>mysql2</code> driver)
Auth	JWT (<code>jsonwebtoken</code>) + <code>bcryptjs</code> , role + per-user permission middleware
File uploads	<code>multer</code> → <code>server/uploads/</code> (served statically at <code>/uploads</code>)
Scheduled jobs	<code>node-cron</code> (booking balance reminder job)
Email	<code>nodemailer</code> (forgot-password OTP)
PDF / print	<code>@react-pdf/renderer</code> for report/invoice PDFs; custom HTML/CSS template for 80mm thermal receipt (“chit”) printing, English + Tamil
Package manager	npm — <code>server/</code> and <code>client/</code> are independent projects, each with its own <code>package.json</code> (no monorepo tool)

2. Project Structure

```

server/
  app.js          # Real entry point (npm run dev / npm start)
  db.js           # Sequelize models + associations
  dotenv-loader.js # Loads server/.env regardless of cwd
  jobs.js         # node-cron reminder job
  routes/         # 13 route modules, mounted under /api/v1/*
  middleware/
    authMiddleware.js # requireAuth – verifies JWT, loads user + role + permissions
    can.js            # can(menuKey, action) – RBAC gate
    audit.js          # writes AuditLog entries
  utils/
    receiptNo.js      # unique receipt/booking number generation
    mailer.js         # nodemailer / OTP email
  uploads/          # logo + QR image uploads (multer target)
  .env              # DB_*, JWT_*, PORT, SMTP_* (see below)

client/
  vite.config.js    # vite-plugin-pwa/Workbox config; dev proxy /api, /uploads → :4000
  src/
    main.jsx, App.jsx # entry + router; routes are permission-guarded via a Guard component
    config.js         # API base URL
    auth/AppContext.jsx # global state: user, menu, lookups, settings, online status, etc.
    api/client.js     # fetch wrapper + token handling
    db/dexie.js       # IndexedDB schema (lookups, settings, outbox, localTx, devTools)
    db/syncEngine.js  # offline outbox flush engine
    pages/            # one file per screen (Pos.jsx, Bookings.jsx, Devotees.jsx, etc.)
    components/       # shared UI: Layout, DataTable/DataGrid, Modal, ConfirmModal, etc.
    print/            # Chit.jsx (thermal receipt), InvoicePDF.jsx, TablePDF.jsx, etc.

```

Note: `server/` also contains a `prisma/` folder and a few root-level files (`auth.js` , `bookings.js` , `users.js` , `can.js` , etc.) that are **not** imported by `app.js` and are not part of the running system. The paths listed above are the ones actually wired in — use those as ground truth when tracing behavior.

3. Getting Started

Database

```
mysql -u root -p -e "CREATE DATABASE tmadj CHARACTER SET utf8mb4;"
```

The schema itself must already exist in the database (see [Database Schema](#)) — on boot, the server only runs `DevoteeMembership.sync({ force: false })`; it does not create the other tables.

Populate/restore the schema and seed data (roles, menu items, default admin user, lookup values) from an existing SQL dump, or via `mysql` scripts, before first run.

Server

```
cd server
npm install
npm run dev          # node app.js → http://localhost:4000
```

Create `server/.env` with:

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=tmadj
DB_USER=root
DB_PASS=your-password
JWT_SECRET=<long random string>
JWT_REFRESH_SECRET=<another long random string>
PORT=4000
SMTP_HOST=...
SMTP_PORT=...
SMTP_SECURE=true|false
SMTP_USER=...
SMTP_PASS=...
SMTP_FROM=...
```

These are read directly by `db.js` / `dotenv-loader.js`. (`server/.env.example` in the repo is stale — it shows a `DATABASE_URL` Prisma-style var that is not actually read; use the `DB_*` variables above instead.)

Client

```
cd client
npm install
npm run dev          # vite → http://localhost:5173, proxies /api and /uploads to :4000
```

Default login

`admin@tmadj.com` / `Admin@123` (Superadmin) — assuming the seed/restore step above created this account. Change the password immediately in production.

Production build

```
cd client && npm run build      # outputs client/dist with service worker
# Serve dist/ behind HTTPS (required for service workers)
# Reverse-proxy /api and /uploads to the Node server
```

Counter device ID

Each physical counter should have a unique device prefix, set once per browser:

```
localStorage.setItem('tmadj.deviceId', 'C1')
```

This prefixes locally-generated offline chit numbers (`C1-0001` , `C1-0002` , ...).

4. Database Schema

All models are defined in `server/db.js` with Sequelize, `timestamps: false`, lowercase `tableName`. Schema evolves via manual SQL patches in `server/migrations/` (e.g. `add_addressTa_to_devotee.sql`) rather than a migration tool.

Model	Table	Purpose
Role	role	Named roles (Superadmin, Manager, Staff, ...)
MenuItem	menuitem	Sidebar pages, keyed by <code>key</code> (e.g. <code>pos</code> , <code>bookings</code>)
User	user	Staff accounts — email/password login, OTP fields for password reset
RolePermission	rolepermission	Per-role View/Create/Edit/Delete/Export flags per <code>MenuItem</code>
UserPermission	userpermission	Per-user overrides of the same flags — takes precedence over role permissions
LookupType	lookuptype	A dropdown/category type (<code>code</code> , e.g. <code>PAYMENT_MODE</code> , <code>HALL_EVENT</code>); <code>isPosMenu</code> marks POS category types
LookupValue	lookupvalue	A value within a type; self-referencing <code>parentId</code> for group/sub-item structure (e.g. Donation main → sub-categories); <code>price</code> , <code>depositPct</code> , <code>meta</code> (JSON, e.g. membership duration)
Devotee	devotee	Member/customer record — contact info, IC, membership fields
DevoteeMembership	devoteemembership	Historical membership records per devotee (the only table Sequelize auto-syncs on boot)
DevoteeNote	devoteenote	Free-text notes on a devotee
Transaction	transaction	A POS sale — <code>clientUuid</code> (idempotency key), <code>receiptNo</code> , <code>localChitNo</code> , <code>kind</code> , <code>total</code> , <code>paymentMode</code> , <code>status</code> , optional <code>bookingId</code> link
TransactionItem	transactionitem	

Model	Table	Purpose
		Line items — <code>nameSnapshot</code> / <code>priceSnapshot</code> freeze the item's name/price at sale time
<code>HallBooking</code>	<code>hallbooking</code>	A hall reservation — <code>bookingNo</code> , event window, <code>hallRate</code> , <code>depositDue</code> , <code>status</code>
<code>BookingNote</code>	<code>bookingnote</code>	Free-text notes on a booking
<code>Reminder</code>	<code>reminder</code>	Scheduled balance-due reminders, picked up by the cron job
<code>Setting</code>	<code>setting</code>	Key-value store (JSON <code>value</code>) for temple profile, receipt config, numbering, message templates, etc.
<code>AuditLog</code>	<code>auditlog</code>	Who did what, when — written by <code>middleware/audit.js</code>

Key associations: `User` `belongsTo` `Role`; `LookupValue` `belongsTo` `LookupValue` as parent (self-referencing); `Transaction` `belongsTo` `Devotee` and `belongsTo` `HallBooking`; `HallBooking` `hasMany` `Transaction` as `transactions` (payment history); `HallBooking` `hasMany` `Reminder`.

5. API Reference

All routes are mounted at `/api/v1/*` in `server/app.js`. Every route module (except `auth`) applies `requireAuth` and most gate individual endpoints with `can(menuKey, action)`.

Base path	Endpoints
/auth	POST /login , POST /refresh , GET /me , POST /forgot-password , POST /verify-otp , POST /reset-password
/lookups	GET / (active values, for app boot), GET /types , POST /types , PUT /types/reorder , PUT /types/:id , DELETE /types/:id , POST /:typeCode/values , PUT /values/reorder , PUT /values/:id , DELETE /values/:id
/menu	GET / — permission-filtered sidebar menu for the logged-in user
/users	GET / , POST / , PUT /:id , DELETE /:id , GET /:id/permissions , PUT /:id/permissions
/roles	GET / , POST / , GET /:id/permissions , PUT /:id/permissions
/devotees	GET /:id , GET / , POST / , PUT /:id , DELETE /:id , POST /:id/memberships , PUT /:devoteeId/memberships/:memId , POST /:id/notes , PUT /:devoteeId/notes/:noteId , DELETE /:devoteeId/notes/:noteId
/transactions	POST / (create sale), GET / (list/filter), PUT /:id , PATCH /:id/cancel
/sync/transactions	POST / — batch, idempotent ingest of offline-queued sales (dedup by <code>clientId</code>)
/bookings	GET /check-conflict , GET /calendar , GET / , POST / , POST /:id/payments , GET /:id , PATCH /:id/notes , POST /:id/booking-notes , PUT /:id/booking-notes/:noteId , DELETE /:id/booking-notes/:noteId , PATCH /:id/cancel
/dashboard	GET /summary
/reports	GET /daily-collection , GET /archanai , GET /donations , GET /annadhanam , GET /payments (all via a shared <code>txReport</code> helper

Base path	Endpoints
	filtered by <code>kind</code>), <code>GET /bookings</code> , <code>GET /devotees</code> , <code>GET /audit</code>
<code>/settings</code>	<code>GET /</code> , <code>PUT /</code> , <code>POST /logo</code> (multer), <code>GET /numbering</code> , <code>POST /numbering</code> , <code>POST /qr-image</code> (multer), <code>GET /dismissals</code> , <code>POST /dismissals</code>
<code>/payments</code>	<code>GET /</code> (unified transaction ledger), <code>GET /membership</code> (membership payments)

CORS allow-list is hardcoded in `server/app.js` (`ALLOWED_ORIGINS`) — add new deployment origins there.

6. Auth & RBAC

- **Login:** `POST /api/v1/auth/login` verifies email/password (bcrypt), returns a JWT access token + refresh token.
- **requireAuth** (`server/middleware/authMiddleware.js`): verifies the JWT, loads the `User` with `role.permissions.menuItem` and `permissions.menuItem` (`UserPermission`) via Sequelize `include` , attaches to `req.user` .
- **can(menuKey, action)** (`server/middleware/can.js`): Superadmin always passes. Otherwise checks `req.user.permissions` (per-user override) first, falling back to `req.user.role.permissions` (role default). `action` is one of `view` , `create` , `edit` , `delete` , `export` , mapped to `canView` / `canCreate` / `canEdit` / `canDelete` / `canExport` on the matching `RolePermission` / `UserPermission` row.
- **Client-side:** `AppContext.jsx` exposes `can(menuKey, action)` (Superadmin bypass, else looks up `user.permissions[menuKey][action]`), and `App.jsx` 's `Guard` component blocks route rendering when the user lacks `view` on that route's `menuKey` . This is a UX convenience only — the API is the actual enforcement point.
- **Audit:** `server/middleware/audit.js` writes `AuditLog` rows for mutating actions on key routes.

7. Offline-First Architecture

Local storage (Dexie / IndexedDB — `client/src/db/dexie.js`)

Database name `tmadj` , current version 5, tables: - `lookups` , `settings` — mirrors of server master data, refreshed on login/online via `AppContext.refreshMasterData()` - `outbox` — pending POS sales (keyed by `clientId`) - `localTx` — local copy of every sale (for the “All Orders” screen to

work offline) - `devotees` , `cachedBookings` — read-only mirrors for offline search/browse - `devoteeOutbox` , `userOutbox` , `bookingOutbox` , `paymentOutbox` — per-resource pending-create/pending-payment queues

Sync engine (`client/src/db/syncEngine.js`)

- `queueTransaction(tx)` — called on every POS sale; writes to `outbox` + `localTx` , then triggers a flush.
- `flush()` — guarded against concurrent runs. Checks `navigator.onLine` and pings `GET /api/v1/health` (4s timeout) to confirm the server is actually reachable, then:
 1. POSTs all pending `outbox` transactions to `POST /sync/transactions` as a batch (idempotent by `clientId`); on success removes from `outbox` , updates `localTx` with the real `receiptNo` and `synced: true` .
 2. Flushes `devoteeOutbox` , `userOutbox` , `bookingOutbox` , `paymentOutbox` one item at a time against their normal REST endpoints.
- Runs on: an `online` browser event, every 10 seconds via `setInterval` , and once immediately on `startSyncEngine()` .
- `onSyncChange(fn)` lets components (e.g. the header sync badge) subscribe to online/pending-count changes.

Snapshotting

`TransactionItem.nameSnapshot` / `priceSnapshot` are copied from the `LookupValue` at sale time, so historical chits and reports never change even if an item's name or price is edited later. Consequently, `LookupValue` deletion is soft where the value has sale history — the delete endpoints deactivate (`isActive: false`) instead of hard-deleting when the value is referenced. The same pattern applies to `HallBooking` / `Transaction` cancellation: status changes to `CANCELLED` , rows are never deleted.

PWA

`client/vite.config.js` configures `vite-plugin-pwa` with Workbox: `NetworkFirst` for all `GET /api/v1/*` calls (4s timeout, falls back to cache), `CacheFirst` for `/uploads/*` assets, and a full app-shell precache for offline navigation.

8. Frontend Architecture

- **Routing** (`client/src/App.jsx`): `BrowserRouter` with a `Guard` wrapper component per route — redirects to `/login` if unauthenticated, and shows an access-denied message if the user lacks `view` permission on that route's `menuKey` . All authenticated routes render inside a shared `Layout` (sidebar + header + sync status).

- **Global state** (`client/src/auth/AppContext.jsx`): holds `user` , `booted` , `lookups` , `settings` , `menu` , `online` , and exposes `login()` , `logout()` , `can()` , `refreshMasterData()` . On boot, tries to fetch fresh master data from the API; on failure (offline), falls back to the Dexie mirror.
 - **API client** (`client/src/api/client.js`): fetch wrapper handling the `Authorization: Bearer` header, JSON body serialization, and (for uploads) `FormData` .
 - **Shared components** (`client/src/components/`): `DataGrid` / `DataTable` (paginated/sortable tables), `Modal` / `ConfirmModal` , `Tabs` , `FilterBar` (+ CSV export helper), `SearchableSelect` , `LookupSelect` (dropdown bound to a `LookupType` code), `DevoteeSearch` (typeahead), `SendMessageModal` (WhatsApp template filling), `Toast` .
 - **Theme**: `client/src/theme/tokens.css` — maroon/saffron/gold/cream “Divya” palette used via Tailwind utility classes throughout.
-

9. Receipt / Chit Printing

- `client/src/print/Chit.jsx` renders the 80mm thermal receipt as HTML/CSS (English + Tamil labels, controlled by the `receipt.language` setting), sized for direct browser printing (`window.print()`).
 - Report and invoice exports use `@react-pdf/renderer` : `client/src/print/InvoicePDF.jsx` (hall booking invoice), `client/src/print/TablePDF.jsx` (generic tabular report export), orchestrated by `printInvoice.js` .
 - Receipt/booking numbering (prefix + digit count) is configurable per-type via `Setting` rows (`numbering.receiptPrefix` , `numbering.receiptDigits` , `numbering.bookingPrefix` , `numbering.bookingDigits`) and generated server-side by `server/utils/receiptNo.js` , which retries with a new random number on collision (up to 20 attempts) rather than using a sequential counter.
-

10. Scheduled Jobs

`server/jobs.js` — `startReminderJob()` , called from `app.js` on server start. Runs daily at 09:00 (`node-cron` schedule `0 9 * * *`): finds `Reminder` rows with `status: 'SCHEDULED'` and `scheduledAt <= now` , logs them, and marks them `SENT` . Actual WhatsApp/SMS delivery is not implemented — this is currently a log-only hook point for wiring in a messaging gateway.

11. File Uploads

`multer` handles two upload endpoints, both under `/api/v1/settings`, writing to `server/uploads/` (served statically at `/uploads`): - `POST /settings/logo` — temple logo (shown in sidebar, login screen, chit header) - `POST /settings/qr-image` — DuitNow QR image (shown full-screen at POS checkout)

12. Build & Deploy

```
cd client && npm run build      # → client/dist (includes service worker; requires HTTPS in prod)
cd server && npm start          # node app.js
```

Serve `client/dist` behind a reverse proxy (nginx or similar) that also proxies `/api` and `/uploads` to the Node server. Add any new production origin to `ALLOWED_ORIGINS` in `server/app.js` before deploying to a new domain.